

Help Menu

The commands are described under the [Help Menu](#).

Methods of the Broker

_Methods of the Broker

The *Broker* provides:

- The *methods* listed in the table of contents to the left.
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
contentsList	[]			([Contents:table]) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				(IconName:string, Width:integer, Height:integer) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method
 Identifier and data type of the parameter you enter
 Data type of the return value

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object

Name of the method
 Identifier and data type of the parameter you enter
 Default value
 Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, *:= false* in the example above.

If the *method* has a return value, the signature shows its data type after the arrow *->*, *→ boolean* in the example above.

brokerStat [SimTalk]

Returns the statistics table of the *Broker* designated by <Path> and writes it into a table.

Remarks

For *Broker hierarchies*, i.e., for several *Brokers* connected with *Connectors*, the brokered services are only considered for the *Broker* which you typed into the *Importer*. This also applies when the service is provided by *sub-Brokers*.

Type**Method****Syntax**

```
<Path>.brokerStat([BrokerStatisticsTable:table]) → boolean
```

Parameter

The optional parameter *BrokerStatisticsTable* of data type *Table* designates the name of the table.

Return Value

The return value has the data type *boolean*.

- true if statistics collection is activated.
false if statistics collection is deactivated. Then the table remains unaltered.
- Is the table containing the statistics data if statistics collection is activated if you do not specify the optional parameter.
void if statistics collection is deactivated.

Examples

```
MyBroker.brokerStat(MyBrokerStatisticsTable)
// Writes the statistics values to the table named MyBrokerStatisticsTable.
// The return value is true if statistics collection is activated.
// It is false if it is deactivated. Then the table remains unaltered.
```

```
MyBroker.brokerStat
// Writes the statistics values to a table.
```

See also

[Tab Statistics \[Broker\]](#)

doStandardExport [SimTalk]

Makes the *Broker* designated by <Path> do a standard export of the specified *Worker/Exporter*.

Remarks

Use *doStandardExport* in the **Exporter Request Control** of the *Broker*.

If you do not want the *Broker* to assign a specific *Worker/Exporter*, then either set *AutomaticMediation* to `false` or do not call the methods *doStandardImport* or *doStandardExport*.

Type

Method

Syntax

```
<Path>.doStandardExport(Exporter:object)
```

Parameter

The parameter *Exporter* of data type *object* designates the *Worker/Exporter*.

Example

```
MyBroker.doStandardExport(MyExporter)
```

See also

[Exporter Request Control \[Broker\]](#)

[AutomaticMediation \[SimTalk\] - Worker](#)

[AutomaticMediation \[SimTalk\] - Exporter](#)

[doStandardImport \[SimTalk\]](#)

doStandardImport [SimTalk]

Makes the *Broker* designated by <Path> do a standard import of the specified *importer*.

Remarks

Use *doStandardImport* in the **Importer Request Control** of the *Broker* for those importers for which you do not need special treatment.

This way you do not have to call the methods *testImportFor* and *engage* for the designated *importer*.

Type

Method

Syntax

```
<Path>.doStandardImport(Importer:object, Type:integer)
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its **type**: 0 designates the *failure/remove failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Examples

```
MyBroker.doStandardImport(MyImporter, 2)
```

```
param obj: object, type: integer // Importer type (0=failure importer,
1=setup importer, 2=processing importer, 3=transport importer)
// look for the worker in the attribute of the part
if @.worker /= void
  // a worker was already assigned to the part
  if @.worker.freeCapacity > 0
    // worker is free, we can assign the worker to the station
    var workersToBeExported : table[object, string, integer]
    workersToBeExported.create
    workersToBeExported.writeRow( 1, workersToBeExported.YDim+1,
@.worker, "StandardService", 1 )
    ?.engage( obj, type, workersToBeExported)
  end
else
  // there is no assignment between the part and any worker
```

```
    ?.doStandardImport(obj, type)  
end
```

SimTalk

[testImportFor \[SimTalk\]](#)

[engage \[SimTalk\]](#)

See also

[Importer Request Control \[Broker\]](#)

engage [SimTalk]

Makes the *Broker* designated by <Path> assign one or several *Exporters/Workers* to an *importer*.

Remarks

The *Broker* attempts to book the specified services and the amount required with the *Exporters/Workers*. The *Exporters/Workers* have to be registered with any *Broker*. *Exporters/Workers* and services may be contained multiple times.

Type

Method

Syntax

```
<Path>.engage(Importer:object, Type:integer, Data:table) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure/remove failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.
- The parameter *Data* of data type *Table* designates a table with the columns **Exporter**, **Services**, and **Amount**.

Return Value

The return value has the data type *boolean*.

true if all services were assigned with the required amount.

false if the services were not assigned. The *Broker* does not save and remember requests it cannot fulfill.

Examples

```
MyBroker.engage(importer1,1,tableTile)
```

```
param obj: object, type: integer
// importer type (0 is the failure, 1 the setup,
//2 the process, and 3 the transport importer)
var tab : table[object, string, integer]
```

```
var service : string
tab.create
tab.writeRow(1,1, .Resources.Worker:1, "myservice", 1 )
return MyBroker.engage(obj, type, tab)
```

forgetOpenRequest [SimTalk]

Cancels an open request with the *Broker* designated by <Path>.

Type

Method

Syntax

```
<Path>.forgetOpenRequest(Importer:object, Type:integer)
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *importer*.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Example

```
MyBroker.forgetOpenRequest(Station,1)
```

See also

[Open Importers \[Broker\]](#)

getExportersForService [SimTalk]

Returns the *Exporters/Workers* who provide the desired service and who can be brokered by the *Broker* designated by <Path>.

Type

Method

Syntax

```
<Path>.getExportersForService(Service:string) → object[]
```

Parameter

The parameter *Service* of data type *string* designates the service.

Return Value

The return value is an *array* of data type *object*.

Example

```
MyBroker.getExportersForService("MyService")
```

getOfferedServices [SimTalk]

Returns all services which are offered by *Exporters/Workers* that are managed by the *Broker* designated by <Path>, and writes them into a table.

Type

Method

Syntax

```
<Path>.getOfferedServices([Services:table]) → table
```

Parameter

The optional parameter *Services* of data type *table* designates the name of the table. The subtable lists all *Exporters/Workers* offering the service.

Examples

```
MyBroker.getOfferedServices(MyOfferedServicesTable)  
// Writes the offered services to the table MyOfferedServicesTable.
```

```
MyBroker.getOfferedServices  
// Writes the offered services to a table which is the return value.
```

See also

[Offered Services \[Broker\]](#)

getOpenImporters [SimTalk]

Returns all open, i.e., unsatisfied *importers* managed by the *Broker* designated by <Path> and writes them into a table.

Type

Method

Syntax

```
<Path>.getOpenImporters([Importers:table]) → table
```

Parameter

The optional parameter *Importers* of data type *table* designates the name of the table.

- Column 1 of data type *object* lists all objects whose *importers* registered an unsatisfied request with the *Broker*.
- Column 2 of data type *integer* designates its type: 0 designates the *failure/remove failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Return Value

The return value has the data type *table*.

Examples

```
MyBroker.getOpenImporters(MyOpenImportersTable)  
// Writes the open importers to the table MyOpenImportersTable.
```

```
MyBroker.getOpenImporters  
// Writes the open importers to a table which is the return value.
```

See also

[Open Importers \[Broker\]](#)

getSatisfiedImporters [SimTalk]

Returns all *importers*, whose request for an *Exporter/Worker* the *Broker* designated by <Path> fulfilled, and writes them into a table.

Type

Method

Syntax

```
<Path>.getSatisfiedImporters([Importers:table]) → table
```

Parameter

The optional parameter *Importers* of data type *table* designates the name of the table.

- Column 1 of data type *object* of the table contains all objects whose *importer* registered a request with the *Broker* and who were assigned an *Exporter/Worker*.
- Column 2 of data type *integer* designates its type: 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Return Value

The return value has the data type *table*.

Examples

```
MyBroker.getSatisfiedImporters(MySatisfiedImporterTable)  
// Writes the satisfied importers to the table MySatisfiedImporterTable.
```

```
MyBroker.getSatisfiedImporters  
// Writes the satisfied importers to a table which is the return value.
```

See also

[SetUpOnlyWhenEmpty \[SimTalk\]](#)

globalOpenRequestsFor [SimTalk]

Finds all *importers* for which the *Exporter/Worker* exports services which are managed by the *Broker* designated by <Path>.

Remarks

globalOpenRequestsFor only works if the *importers* registered their request with a *Broker* in the *Broker* hierarchy.

As opposed to the method *localOpenRequestsFor*, which only checks the *importers* locally managed by the *Broker*, the method *globalOpenRequestsFor* checks the entire *Broker* hierarchy.

Type

Method

Syntax

```
<Path>.globalOpenRequestsFor(Exporter:object, Importers:table)
```

Parameters

- The parameter *Exporter* of data type *object* designates the *Exporter/Worker*.
- The parameter *Importers* of data type *table* contains those *importers* which require at least one service that is exported by the *Exporter/Worker*.

Example

```
MyBroker.globalOpenRequestsFor(Exporter, tab2)  
MyBroker.globalOpenRequestsFor(Worker , tab2)
```

SimTalk

localTestImportFor [SimTalk]

globalTestImportFor [SimTalk]

Checks all *Exporters/Workers* that may be reached in the *Broker* hierarchy of the *Broker* designated by <Path> for the *importer*.

Remarks

As opposed to the method *emlocalTestImportFor*, which only checks the *Exporters/Workers* locally managed by the *Broker*, the method *globalTestImportFor* checks the entire *Broker* hierarchy, i.e., all *Brokers* linked with a *Connector*.

The methods *localTestImportFor* and *globalTestImportFor* return the same result for a single *Broker*!

Type

Method

Syntax

```
<Path>.globalTestImportFor(Importer:object, Type:integer, Exporters:table[,  
OnlyAvailable:boolean]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.
- The parameter *Exporters/Workers* of data type *table* contains all *Exporters/Workers* directly managed by the *Broker* and able to provide any service the *importer* requires. However, the *Broker* does not check if the *Exporters/Workers* are able to export the service or are available.
- The optional parameter *OnlyAvailable* of data type *boolean* sets if all *Exporters/Workers* will be listed (*false*) or if only the actually available *Exporters/Workers* will be listed (*true*).

Return Value

The return value has the data type *boolean*.

The *Broker* does not check if the *Exporters/Workers* can export the service or are available.

Example

```
MyBroker.globalTestImportFor(sp2, 2, tab1)
```

SimTalk

[localTestImportFor \[SimTalk\]](#)

localOpenRequestsFor [SimTalk]

Finds all *importers* for which the *Exporter/Worker* exports services which are managed by the *Broker* designated by <Path>.

Remarks

This only works if the *importers* registered their request with a *Broker* in the *Broker* hierarchy.

Type

Method

Syntax

```
<Path>.localOpenRequestsFor(Exporter:object, Importers:table)
```

Parameters

You can specify the following parameters:

- The parameter *Exporter* of data type *object* designates the *Exporter/Worker*.
- The parameter *Importers* of data type *table* contains those *importers* which require at least one service that is exported by the *Exporter/Worker*.

Example

```
MyBroker.localOpenRequestsFor(Exporter, tab)
```

SimTalk

[globalOpenRequestsFor \[SimTalk\]](#)

localTestImportFor [SimTalk]

Checks all *Exporters/Workers* directly managed by the *Broker* designated by <Path> for the *Importer*.

Type

Method

Syntax

```
<Path>.localTestImportFor(Importer:object, Type:integer, Exporters:table[,  
OnlyAvailable:boolean]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *Importer* proper.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.
- The parameter *Exporters* of data type *table* contains all *Exporters/Workers* directly managed by the *Broker* and able to provide any service the *Importer* requires. However, the *Broker* does not check if the *Exporters/Workers* are able to export the service or are available.
- The optional parameter *OnlyAvailable* of data type *boolean* sets if all *Exporters/Workers* will be listed (*false*) or if only the actually available *Exporters/Workers* will be listed (*true*). This parameter only prevents paused or failed *Exporters/Workers* from being returned.

Return Value

The return value has the data type *boolean*.

Example

```
MyBroker.localTestImportFor(ParallelStation3,0,tab3)  
MyBroker.localTestImportFor(pp3,0,tab3,true)
```

SimTalk

globalTestImportFor [SimTalk]

serviceStat [SimTalk]

Returns *Service Statistics* of the *Broker* designated by <Path, and writes it into a table.

Remarks

If a service is fulfilled by several *Workers/Exporters*, *Plant Simulation* records this time interval only once. The same applies for the count.

For the time during which the *Worker* was en-route to his job, statistics counts the time of the *Worker* who was en-route the longest.

Type

Method

Syntax

```
<Path>.serviceStat([ServiceStatistics:table]) → boolean
```

Parameter

The optional parameter *ServiceStatistics* of data type *table* designates the name of the table.

Item English	Description	Read-only attribute	Item German
Services	Shows the services, which the <i>Broker</i> brokered.	—	Dienste
Dwelling Time: Count	Shows how often the service in this row was brokered, i.e., how often the <i>Exporter/Worker</i> stayed at the <i>importer/importers</i> .	—	Verweildauer: Anzahl
Sum	Shows the sum of the times during which the service in this row was brokered.	StatStayTime [SimTalk]	Summe
Mean Value	Shows the mean duration of the times during which the service in this row was brokered.	StatStayTimeMu [SimTalk]	Mittelwert
Standard Deviation	Shows the standard deviation from the mean value of the times during which the service in this row was brokered.	StatStayTimeDelta [SimTalk]	Standardabweichung
Min	Shows the minimum time during which the service in this row was brokered.	—	Min
Max	Shows the maximum time during which the service in this row was brokered.	—	Max

Item English	Description	Read-only attribute	Item German
Mediation Time: Count	Shows how often the <i>importer/importers</i> were waiting for the procurement of the service in this row.	—	Vermittlungsdauer: Anzahl
Sum	Shows the sum of the times during which the <i>importer/importers</i> were waiting for the procurement of the service in this row.	StatMediationTime [SimTalk]	Summe
Mean Value	Shows the mean duration of the times during which the <i>importer/importers</i> were waiting for the procurement of the service in this row.	StatMediationTimeMu [SimTalk]	Mittelwert
Standard Deviation	Shows the standard deviation from the mean value of the times during which the <i>importer/importers</i> were waiting for the procurement of the service in this row.	StatMediationTimeDelta [SimTalk]	Standardabweichung
Min	Shows the minimum time during which the <i>importer/importers</i> were waiting for the procurement of the service in this row.	—	Min
Max	Shows the maximum time during which the <i>importer/importers</i> were waiting for the procurement of the service in this row.	—	Max

Return Value

The return value has the data type *boolean*.

- true if statistics collection is activated.
false if statistics collection is deactivated. Then the table remains unaltered.
- Is the table containing the statistics data if statistics collection is activated if you do not specify the optional parameter.
void if statistics collection is deactivated.

Examples

```
MyBroker.serviceStat(MyServicesTable)
// Writes the statistics values of the Broker to the table named
MyServicesTable.
```

```
// The return value is true if statistics collection is activated.  
// It is false if it is deactivated. The table then remains unchanged.
```

```
MyBroker.serviceStat  
// Writes the statistics values to a table.  
// The return value is the statistics table if statistics collection is  
activated.  
// It is void if statistics collection is deactivated.
```

See also

[Service Statistics \[Broker\]](#)

Statistics report, [Dwelling Time](#)

Statistics report, [Mediation Time](#)

testImportFor [SimTalk]

Executes a test request for the *Importer*. The return value shows if the *Broker* designated by <Path> is able to satisfy the request (true) or not (false).

Remarks

Use *testImportFor* to find out how the *Broker* will treat the request of the *importer*.

Type

Method

Syntax

```
<Path>.testImportFor(Importer:object, Type:integer, Exporter:table) →  
boolean
```

Parameters

You can specify the following parameters:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.
- The parameter *Exporter* of data type *table* passes a table containing all *Exporters/Workers* that may be assigned after the call.

Return Value

The return value has the data type *boolean*.

Example

```
MyBroker.testImportFor(sp1,1,Tab)
```

SimTalk

[globalTestImportFor \[SimTalk\]](#)

[localOpenRequestsFor \[SimTalk\]](#)

Read-Only Attributes of the Broker

_Read-Only Attributes of the Broker

The *Broker* provides:

- The *read-only attributes* listed in the table of contents to the left.
- The [_Read-Only Attributes of All Objects](#).

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
NumPartsSinceSetup	0			-> integer
NumPred	1			-> integer
NumSucc	3			-> integer

Name of the read-only attribute Current value of the read-only attribute Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

To query the value of a *read-only attribute*, you might, for example, type:

```
print MyBroker.OpenCapacity
```